HANDBUCH

Lochstreifen-Hilfsprogramme

Anzeige, Eingabe und Umwandlung virtueller Lochstreifen: Format, Aufruf, Zwischenablage und Installation

Dieses PDF gibt die Quelle im Wortlaut wieder; Listen, Tabellen und Codeblöcke entsprechen der Quelle, weich umbrochene Zeilen sind zu Absätzen zusammengeführt.

Quelle	`C8205-Interpreter-Lochstreifen-Hilfsprogramme.md`	0.95-001
Stand	21.10.2026	
Umfang	10 Seiten, A4 hoch	
Erzeugt mit	`erzeuge_lochstreifen_hilfsprogramme_pdf.py`	

> Maßgeblich bleibt die Quelldatei. Dieses PDF wird nicht automatisch nachgeführt: Es ist aktuell, solange die Version oben mit der Versionszeile der Quelldatei übereinstimmt.

C 8205 – Lochstreifen-Hilfsprogramme

Quelle: C8205-Interpreter-Lochstreifen-Hilfsprogramme.md · im Wortlaut der Quelle

Inhaltsverzeichnis

1. Zweck . . . 4

2. Format des virtuellen Lochstreifens . . . 4

3. Verfahren . . . 4

3.1 Anzeige (c8205_lochstreifen_anzeige.py) . . . 4
3.2 Eingabe (c8205_lochstreifen_eingabe.py) . . . 5
3.3 Kopieren in die Zwischenablage (Anzeige und Eingabe) . . . 5
3.4 Umwandlung (c8205_lochstreifen_umwandlung.py) . . . 5

4. Benutzung . . . 6

4.1 Anzeige . . . 6
4.2 Eingabe . . . 7
4.3 Umwandlung . . . 8
4.4 Typischer Ablauf . . . 9

5. Technische Voraussetzungen (Installation siehe 5a) . . . 9

5a. Installation der benötigten Programme . . . 9

Linux . . . 9
Windows . . . 10
macOS . . . 10
WSL (Windows-Subsystem für Linux) . . . 10
Virtuelle Maschinen (Abgleich der Zwischenablage mit dem Host) . . . 10

6. Test unter Windows (noch offen) . . . 10

C 8205 – Lochstreifen-Hilfsprogramme

Version: 0.95-001

Dokumentversion s. oben; **Programmversion 1.9** für Anzeige und Eingabe (Stand 10.10.2026), **Programmversion 1.2** für die Umwandlung (Stand 20.10.2026). Die Programme tragen ihre eigene Versionsnummer (`--version`), unabhängig vom Schema `y.xx-zzz` der Dateien (`C8205-Versionierung.md`, Abschnitt 2).

Programm	Datei	Version
Lochstreifen ausschreiben (Anzeige)	`c8205_lochstreifen_anzeige.py`	1.9
Lochstreifen erzeugen (Eingabe)	`c8205_lochstreifen_eingabe.py`	1.9
Textdatei in Lochstreifen umwandeln und umgekehrt (Umwandlung)	`c8205_lochstreifen_umwandlung.py`	1.2

Die Versionsnummer lässt sich mit `--version` abfragen.

Änderungen

- **Umwandlung 1.2**: Die Schalter heißen jetzt `--LS-LESER` und `--LS-STANZER` (bisher `--LESER` und `--STANZER`), wie bei Anzeige und Eingabe. Die früheren Schreibweisen werden weiterhin angenommen. Die Fehlermeldungen nennen die neuen Namen (Abschnitte 3.4 und 4.3).
- **Umwandlung 1.1**: Neue Schalter `--IN-BIN` (Vorgabe) und `--IN-TXT` wählen die Richtung. `--IN-TXT` wandelt einen binär codierten Lochstreifen in eine Textdatei zurück (Abschnitte 3.4 und 4.3).
- **Umwandlung 1.0**: Erste Fassung des dritten Hilfsprogramms (Abschnitte 3.4 und 4.3). Wandelt eine Textdatei ohne Fenster und ohne Tastatureingabe in einen virtuellen Lochstreifen um; unter Linux und Windows lauffähig, nur Python-Standardbibliothek (kein Tkinter).
- **1.9**: `Um. KB` / `Um. GB` werden in Anzeige und Eingabefenster **nicht mehr ausgegeben**, sie wirken nur (Zeichenlage). „Irrung Zeichen“ löscht das letzte Zeichen der Zeile, „Irrung Satz“ die ganze bisherige Zeile; beide erscheinen nicht als Marke. Zeilenumbrüche werden dadurch nie zurückgenommen. Auf dem Lochstreifen selbst bleiben alle Lochungen unverändert erhalten (Eingabe stanzt sie weiterhin).
- **1.8**: Unter X11 wird zuerst `xsel --clipboard --input` benutzt, `xclip` nur noch, wenn `xsel` fehlt oder scheitert. Grund: In einer VMware-Umgebung kam der Text von `xclip` und von Tk nicht in der VM an, der Weg über `xsel` (vim: `:w !xsel -b`) dagegen schon.
- **1.7**: Neue Option `--zwischenablage auto|tk|system` zur Wahl des Kopierverfahrens (Vorgabe `auto`). `tk` benutzt nur die Tk-Zwischenablage, `system` nur Systemprogramm bzw. Win32-API. Gedacht für Umgebungen mit eigenem Zwischenablage-Abgleich (z. B. VMware), in denen sich Verfahren unterschiedlich verhalten.
- **1.6**: `xclip` wird wieder mit `-t UTF8_STRING` aufgerufen. Ohne Angabe bietet `xclip` den Text nur als `STRING` an; auf einer virtuellen Maschine übernahm der Zwischenablage-Abgleich zum Host solche Inhalte nicht (Terminal-Kopieren kam an, `echo … | xclip -selection clipboard` nicht).
- **1.5**: Derselbe Text wird innerhalb von 5 Sekunden nicht erneut in die Zwischenablage gelegt (Maus loslassen und Strg+C kurz nacheinander). Grund: Auf virtuellen Maschinen verarbeitet das Gast-Zusatzprogramm für den Zwischenablage-Abgleich mit dem Host jeden Besitzerwechsel einzeln; viele Wechsel hintereinander können den Abgleich stören.
- **1.4**: Unter Linux/macOS/WSL wird zuerst das Systemprogramm benutzt (`xclip -selection clipboard -t UTF8_STRING`); die Tk-Zwischenablage ist nur noch Rückfall, wenn kein Programm vorhanden ist oder es scheitert. Grund: Tk und `xclip` kämpften nach einem Test um die Besitzerrolle der Zwischenablage, im Programm kam beim Einfügen nichts an, obwohl der Terminalaufruf funktionierte.
- **1.3**: (Der Aufruf von `xclip` mit `-t UTF8_STRING` wurde in 1.4 wieder zurückgenommen.) bei `--debug` folgt nach dem Kopieren eine Gegenprobe (`xclip -o`/`xsel`), die zeigt, was wirklich in der Zwischenablage liegt, samt angebotenen Zielen.
- **1.2**: Das Kopierverfahren wird **bei jedem Kopieren dynamisch** nach dem laufenden System gewählt (Windows: Win32-API; Linux/macOS/WSL: Tk **und zusätzlich** ein Systemprogramm wie `wl-copy`, `xclip`, `xsel`, `pbcopy`, `clip.exe`, sofern vorhanden). Hintergrund: Die reine Tk-Zwischenablage geht unter X11/Wayland (XWayland) zeitweise verloren. Die Titelleiste nennt das erfolgreiche Verfahren, z. B. „kopiert (12 Zeichen, Tk+xclip)“.
- **1.1**: Markieren und Kopieren in die Zwischenablage jetzt unter Linux und Windows (unter Windows über die Win32-API, mit Wiederholungsversuchen und Rückfall auf Tk). Strg+C, Strg+A und Strg+Einfg werden unter Windows auch über den Tastenkode erkannt. Eingabe: AltGr-Zeichen (`@ { } [ ] \ ~ |`) werden unter Windows nicht mehr als Strg-Befehl behandelt.

- **1.0**: Erste Fassung (unter Linux getestet).

> **Hinweis zum Teststand:** Version 1.2 ist unter Linux nur mit Attrappen der Systemprogramme geprüft, die Fenster selbst nicht automatisch. Der Windows-Zweig wurde noch **nicht** auf einem Windows-Rechner getestet (siehe Abschnitt „Test unter Windows“).

---

1. Zweck

Die drei Programme sind Hilfsmittel zum C8205-Interpreter `c8205_interp.py`. Sie arbeiten mit dem **virtuellen Lochstreifen**, wie ihn der Interpreter an den Eingängen `--leser1` / `--leser2` liest und am Stanzer (`--stanzer1`) schreibt.

- **Anzeige** liest einen virtuellen Lochstreifen und schreibt ihn als lesbaren Text in einem scrollbaren Fenster aus (entspricht dem Fenster „SW Ausgabe“ von `--SW-TERMINAL` des Interpreters; beide Fenster bleiben bis zum Schließen mit X offen).
- **Eingabe** ist das Gegenstück: Sie erzeugt einen virtuellen Lochstreifen aus Tastatureingaben in einem Fenster.
- **Umwandlung** erzeugt einen virtuellen Lochstreifen aus einer fertigen **Textdatei** (`--IN-BIN`, Vorgabe) oder macht aus einem Lochstreifen wieder eine Textdatei (`--IN-TXT`), ohne Fenster und ohne Handeingabe. Sie eignet sich für längere Texte und für Aufrufe aus Skripten.

Der Text in den Fenstern von Anzeige und Eingabe kann mit der Maus markiert und in die Zwischenablage kopiert werden. Die Umwandlung hat kein Fenster.

2. Format des virtuellen Lochstreifens

- Eine Lochungszeile je Textzeile.
- 8 Zeichen `0`/`1` in der Spurfolge `w v u p 8 4 2 1` (`1` = gelocht).
- Leerzeilen werden beim Lesen übersprungen.
- Zeichenvorrat und Lochungen nach **Kodetabelle 1** (Spalte „Ausgabe-Adresse“), identisch zu `TABELLE1` in `c8205_interp.py`.
- Gültige Zeichen haben **ungerade** Parität (ungerade Zahl gelochter Spuren). Ausnahme: „Irrung Zeichen“ (alle 8 Spuren).

3. Verfahren

3.1 Anzeige (`c8205_lochstreifen_anzeige.py`)

1. Der Lochstreifen (Datei oder Verzeichnis) wird gelesen. Bei einem Verzeichnis werden alle Dateien ohne führenden Punkt in Namensreihenfolge hintereinander gelesen; zwischen den Dateien erscheint eine graue Trennzeile `--- Datei: NAME ---`.
2. Jede Zeile wird anhand der Kodetabelle in ein Zeichen übersetzt. Die **Zeichenlage** (Klbuch/Gbuch) wird durch die Zeichen `Um. KB` / `Um. GB` umgeschaltet und gilt auch über Dateigrenzen hinweg. Anfangslage ist KB (mit `--lage GB` änderbar).
3. Darstellung:
   - Normale Zeichen erscheinen als Text. SatzMarke/WRZL ergibt einen Zeilenumbruch, Tabulator einen Tabulator, Leertaste/ZR ein Leerzeichen.
   - `Um. KB`/`Um. GB` erscheinen nicht, sie schalten nur die Zeichenlage. `Irrung Zeichen` löscht das vorige Zeichen der Zeile, `Irrung Satz` die ganze Zeile. Sonstige Zeichen ohne druckbare Entsprechung erscheinen als **blaue Marke**, z. B. `<Transport (Relikt)>`.
   - Zeilen ohne Datenlochung (Vorspann, `00000000`) werden übersprungen.
   - Zeilen mit gerader Zahl gelochter Spuren (außer „Irrung Zeichen“) erscheinen als **rote Marke** `<Paritätsfehler 01010101>`; unbekannte Lochungen als `<unbekannte Lochung …>`; Zeilen, die keine 8 Zeichen `0`/`1` sind, als `<ungültige Zeile …>`.
4. Die Ausgabe erscheint in einem Tkinter-Fenster (130 × 36 Zeichen, Bildlaufleiste, Standardposition rechts unten), das bis zum Schließen mit X offen bleibt. Ohne Fenstermöglichkeit oder mit `--text` geht die Ausgabe auf das Terminal.

3.2 Eingabe (`c8205_lochstreifen_eingabe.py`)

1. Es öffnet sich ein gleichartiges Fenster. Jedes eingegebene Zeichen des R300-Kodes wird **sofort** als Lochungszeile in die Lochstreifendatei geschrieben. Eine vorhandene Datei wird überschrieben, fehlende Ordner werden angelegt.
2. Der Stanzer-Kode kennt die Zeichenlage nicht. Das Programm stanzt `Um. KB` bzw. `Um. GB` selbst, sobald ein Zeichen die andere Lage braucht (Anfang: KB), zeigt es aber nicht an.
3. Eine Lochung lässt sich nicht zurücknehmen (wie am echten Stanzer). Korrektur durch Stanzen von „Irrung Zeichen“ bzw. „Irrung Satz“; im Fenster verschwindet dabei das letzte Zeichen bzw. die Zeile.
4. Zeichen, die nicht im R300-Kode vorkommen, werden abgewiesen (Signalton, Hinweis in der Titelleiste); es wird nichts gestanzt.
5. Das Textfeld ist nicht editierbar; es zeigt nur, was gestanzt wurde.

3.3 Kopieren in die Zwischenablage (Anzeige und Eingabe)

Das Fenster „SW Ausgabe“ des Interpreters (`--SW-TERMINAL`, `c8205_interp.py` ab Version 0.03-007) kopiert auf dieselbe Weise; das Kopierverfahren wählt dort der Schalter `--zwischenablage auto|tk|system` (wie hier).

Kopiert wird

- sofort beim **Loslassen der Maustaste** nach dem Markieren (auch nach Doppelklick),
- mit **Strg+C**, **Strg+Umschalt+C** oder **Strg+Einfg** (auch wenn der Tastaturfokus nicht im Textfeld liegt),
- über das **Kontextmenü** (rechte Maustaste; auf macOS auch mittlere Taste bzw. Strg+Klick) mit „Kopieren“ und „Alles markieren“.

Strg+A markiert den gesamten Text. Jedes erfolgreiche Kopieren meldet sich kurz in der Titelleiste („kopiert (N Zeichen)“), ein Fehlschlag mit „Kopieren fehlgeschlagen“.

Technisch je nach System:

System	Verfahren
Linux, macOS, WSL	zuerst ein Systemprogramm, das die Zwischenablage dauerhaft übernimmt (Tk nur als Rückfall, `clipboard_clear`, `clipboard_append`, `update`, bis zu 5 Versuche): Wayland `wl-copy`, X11 `xsel` (sonst `xclip`), macOS `pbcopy`, WSL `clip.exe`. Welches, wird bei jedem Kopieren nach `WAYLAND_DISPLAY/XDG_SESSION_TYPE/DISPLAY` und den installierten Programmen entschieden; fehlende Programme werden übersprungen
Windows	Win32-API (`user32/kernel32`, Format `CF_UNICODETEXT`, Zeilenenden `\r\n`), bis zu 20 Versuche im Abstand von 50 ms, falls ein anderes Programm die Zwischenablage kurz sperrt; bei Fehlschlag Rückfall auf die Tk-Zwischenablage

Unter Windows werden Strg+C, Strg+A und Strg+Einfg zusätzlich über den Tastenkode erkannt, damit sie auch bei nicht-lateinischem Tastaturlayout wirken.

Nur die **Eingabe** kann außerdem einfügen: Strg+V oder Umschalt+Einfg stanzt die Zeichen aus der Zwischenablage.

3.4 Umwandlung (`c8205_lochstreifen_umwandlung.py`)

Die Umwandlung arbeitet in zwei Richtungen, gewählt mit `--IN-BIN` (Vorgabe) oder `--IN-TXT`. Beide Schalter zusammen sind ein Fehler (Rückgabewert 2). `--LS-LESER` ist immer die **Eingabe**, `--LS-STANZER` immer die **Ausgabe**; was darin liegt, hängt von der Richtung ab:

	`--IN-BIN` **(Vorgabe)**	`--IN-TXT`
Richtung	Text → binär codierter R300-Lochstreifen	binär codierter R300-Lochstreifen → Text
`--LS-LESER`	Textdatei	Lochstreifen
`--LS-STANZER`	Lochstreifen	Textdatei

Die Punkte 1 bis 8 beschreiben `--IN-BIN`; die Rückrichtung steht danach unter „`--IN-TXT`“.

1. **Eingang (`--LS-LESER`)**: eine Textdatei. Ist `--LS-LESER` ein Verzeichnis, werden alle Dateien ohne führenden Punkt in Namensreihenfolge hintereinander umgewandelt (wie bei der Anzeige und beim Leser des Interpreters; die Zeichenlage läuft über die Dateigrenze weiter). Die Datei wird als UTF-8 gelesen (eine UTF-8-Markierung am Dateianfang, BOM, wird entfernt).

2. **Ausgang (`--LS-STANZER`)**: der zu erzeugende Lochstreifen, Datei oder Verzeichnis (dann `VERZ/stanzer-ausgabe.txt`, wie beim Stanzer des Interpreters). Eine vorhandene Datei wird überschrieben, fehlende Ordner werden angelegt. Liegt die Ausgabedatei im `--LS-LESER`-Verzeichnis, wird sie nicht als Eingabe gelesen; sind beide dieselbe Datei, bricht das Programm ab.
3. **Beide Angaben sind Pflicht.** Fehlt eine, meldet das Programm den Fehler und beendet sich mit Rückgabewert 2; es wird nichts geschrieben.
4. **Nur R300-Zeichen**: Verarbeitet werden nur Zeichen der Kodetabelle 1 (Spalte „Ausgabe-Adresse“, siehe `C8205-Interpreter-Kodetabellen.md`). Jedes andere Zeichen (z. B. `ä`, `@`, `{`, `$`, Steuerzeichen) wird **übersprungen**, also nicht gestanzt und nicht ersetzt; das Programm meldet auf der Fehlerausgabe, welche Zeichen es waren, wie oft und wo zuerst (Datei, Zeile). Der Rückgabewert bleibt dann 0.
5. **Zeichenlage**: Im Text steht keine Lage. Sie folgt aus dem Zeichen. Klbuch (KB): Ziffern `0–9`, Kleinbuchstaben `a–z` (ohne `ä ö ü ß`) sowie `% : + / , . -` und der Gravis (Akzent `` ` ``). `Gbuch (GB): Großbuchstaben` A–Z `sowie` < " ≈ [> ' ≋ ~] # * (= ;) ? ! △. `Das Programm stanzt` Um. KB `bzw.` Um. GB `selbst, sobald ein Zeichen die andere Lage braucht. Die Anfangslage ist KB, am Anfang wird also nur vor einem Gbuch-Zeichen` Um. GB `gestanzt (wie bei der Eingabe). Beispiel:` Hallo 1 `ergibt` Um. GB, H, Um. KB, a, l, l, o, `Leertaste`, 1`.
6. **Lageunabhängige Zeichen**: Zeilenende (`\n`, auch `\r\n`) ergibt SatzMarke/WRZL, Tabulator ergibt Tabulator, Leerzeichen ergibt Leertaste/ZR. Sie ändern die Zeichenlage nicht. Das Wagenrücklaufzeichen `\r` allein wird ignoriert, damit Textdateien von Windows und Linux dasselbe Ergebnis liefern.
7. **Nicht erzeugt** werden `Irrung Zeichen`, `Irrung Satz` und `Transport (Relikt)` (im Text gibt es kein Zeichen dafür; Irrungen sind Sache der Handeingabe) sowie Vorspann, Nachspann und Leerzeilen. Jede Lochungszeile ist ein Datenzeichen mit ungerader Parität.
8. **Ausgabeformat** wie in Abschnitt 2: eine Zeile je Zeichen, 8 Ziffern `0`/`1` in der Spurfolge `w v u p 8 4 2 1`, Zeilenende `\n` (auch unter Windows). Das Bitmuster ist die Ausgabe-Adresse der Kodetabelle (z. B. `1` = 001 oktal = `00000001`, `Q` = `Um. GB` `00101111`, dann 130 oktal = `01011000`).

`--IN-TXT` **(Lochstreifen → Text)** arbeitet wie die Anzeige, schreibt aber in eine Textdatei (UTF-8, Zeilenende `\n`):

- `--LS-LESER` ist ein Lochstreifen (Datei oder Verzeichnis, Format nach Abschnitt 2); die Zeichenlage läuft über Dateigrenzen weiter, Anfangslage KB.
- `Um. KB`/`Um. GB` erscheinen nicht, sie schalten nur die Lage. `Irrung Zeichen` löscht das letzte Zeichen der aktuellen Zeile, `Irrung Satz` die ganze bisherige Zeile; ein schon abgeschlossenes Zeilenende wird nie zurückgenommen.
- SatzMarke/WRZL ergibt ein Zeilenende, Tabulator einen Tabulator, Leertaste/ZR ein Leerzeichen; die Zeichen `≈ ≋ △` erscheinen als Unicode-Zeichen.
- Zeilen ohne Datenlochung (`00000000`, Vorspann) werden stillschweigend übersprungen. Ungültige Zeilen (keine 8 Zeichen `0`/`1`), Zeilen mit gerader Lochzahl (Paritätsfehler, außer `Irrung Zeichen`), unbekannte Lochungen und `Transport (Relikt)` werden übersprungen und als Warnung gemeldet (höchstens 10 einzeln).
- Hin und zurück: Text → `--IN-BIN` → `--IN-TXT` liefert den Ausgangstext, sofern er nur R300-Zeichen enthält (geprüft mit 3000 zufälligen Zeichen; `\r\n` kommt als `\n` zurück).

Die Zeichentabelle des Programms ist eine Kopie der Spalten Klbuch, Gbuch und Ausgabe-Adresse aus `TABELLE1` in `c8205_interp.py`. Ändert sich die Tabelle dort, ist die Kopie im Programm (`ZEICHEN_TABELLE`, `LAGEFREI`, `UM_KB`, `UM_GB`) nachzuführen.

4. Benutzung

4.1 Anzeige

```
python3 c8205_lochstreifen_anzeige.py --LS-LESER PFAD [--lage KB|GB] [--text] [--debug] [--version]
```

Option	Bedeutung
`--LS-LESER PFAD`	**Pflicht.** Virtueller Lochstreifen: Datei oder Verzeichnis
`--lage KB|GB`	Zeichenlage am Anfang (Vorgabe `KB`)
`--text`	Ausgabe auf dem Terminal statt im Fenster (auch Rückfall, wenn kein Fenster möglich ist)
`--zwischenablage auto|tk|system`	Kopierverfahren (Vorgabe `auto`): `tk` nur Tk-Zwischenablage, `system` nur Systemprogramm (`xclip` usw.) bzw. Win32-API
`--debug`	Diagnose auf der Fehlerausgabe: Tasten, Mausereignisse, Kopiervorgänge (inkl. verwendetes Verfahren `Win32`/`Tk`)
`--version`	Versionsnummer ausgeben

Beispiele:

```
python3 c8205_lochstreifen_anzeige.py --LS-LESER stanzer-ausgabe.txt
python3 c8205_lochstreifen_anzeige.py --LS-LESER lochstreifen/ --lage GB
python3 c8205_lochstreifen_anzeige.py --LS-LESER band.txt --text > band_klartext.txt
```

Rückgabewert 2 bei einem Lese- oder Dateifehler.

4.2 Eingabe

```
python3 c8205_lochstreifen_eingabe.py --LS-STANZER PFAD [--text] [--debug] [--version]
```

Option	Bedeutung
`--LS-STANZER PFAD`	**Pflicht.** Zu erzeugender Lochstreifen. Datei, oder Verzeichnis (bzw. Pfad mit `/` am Ende): dann `VERZ/stanzer-ausgabe.txt`
`--text`	Zeichen von der Standardeingabe lesen statt im Fenster (auch Rückfall); Ende mit Strg+D (Windows: Strg+Z, Enter)
`--zwischenablage auto|tk|system`	Kopierverfahren wie bei der Anzeige
`--debug`	Diagnose auf der Fehlerausgabe
`--version`	Versionsnummer ausgeben

Beispiele:

```
python3 c8205_lochstreifen_eingabe.py --LS-STANZER eingabe.txt
python3 c8205_lochstreifen_eingabe.py --LS-STANZER lochstreifen/
echo "hallo" | python3 c8205_lochstreifen_eingabe.py --LS-STANZER eingabe.txt --text
```

Unter Windows ggf. `py` oder `python` statt `python3` verwenden.

Tasten im Eingabefenster:

Taste	Wirkung
Buchstaben, Ziffern, Satzzeichen des R300-Kodes	Zeichen stanzen (ggf. mit vorausgehendem `Um. KB`/`Um. GB`)
Leertaste	Leertaste/ZR
Tab	Tabulator
Enter	SatzMarke/WRZL
F1 / F2	`Um. KB` / `Um. GB` von Hand stanzen
Rück-Taste	„Irrung Zeichen“ (alle 8 Spuren)
Entf	„Irrung Satz“
Strg+V, Umschalt+Einfg	Zwischenablage stanzen
Strg+C, Strg+Einfg, Maus loslassen	Markierten Text kopieren

Tippbare Sonderzeichen: Klbuch-Lage `% / + : . , -`, der Gravis `` ` `` sowie Ziffern; Gbuch-Lage < " [> ' ~] # * (= ;) ? ! und die Glyphen ≈ ≋ △`.

Eingabe beenden und speichern. Es gibt kein „Speichern“ und keine eigene Beenden-Taste: Jedes gestanzte Zeichen wird **sofort** in die Lochstreifendatei geschrieben und gleich übergeben, die Datei ist also zu jedem Zeitpunkt auf dem Stand des Fensters. Beendet wird so:

1. **Fenstermodus (Vorgabe):** Das Fenster mit **X** schließen. Dabei wird die Datei ordentlich geschlossen, im Terminal erscheint `Lochstreifen: <Pfad>`. Das Programm endet erst mit dem Schließen des Fensters.
2. **Mit `--text`:** Ende mit **Strg+D** (Windows: **Strg+Z**, dann Enter). Danach wird die Datei geschlossen und der Pfad ausgegeben.
3. Wurde das Programm im Terminal gestartet und lässt sich nicht regulär beenden, bricht **Strg+C** im Terminal es ab; die bis dahin gestanzten Zeilen stehen bereits in der Datei.

Vor dem Beenden beachten:

- Soll der Streifen mit einem Zeilenende (SatzMarke/WRZL) enden, vorher **Enter** drücken; ein Zeilenende wird nur auf Tastendruck gestanzt.
- Die Datei wird beim **Start** neu angelegt: eine vorhandene Datei gleichen Namens wird überschrieben. Einen fertigen Streifen daher nicht erneut mit demselben `--LS-STANZER`-Pfad starten, sondern vorher sichern oder einen anderen Pfad wählen.

4.3 Umwandlung

```
python3 c8205_lochstreifen_umwandlung.py --LS-STANZER PFAD --LS-LESER PFAD [--IN-BIN | --IN-TXT] [--version]
```

Option	Bedeutung
`--LS-STANZER PFAD`	**Pflicht.** Ausgabe: bei `--IN-BIN` der zu erzeugende Lochstreifen, bei `--IN-TXT` die zu erzeugende Textdatei. Datei, oder Verzeichnis (bzw. Pfad mit `/` am Ende): dann `VERZ/stanzer-ausgabe.txt`
`--LS-LESER PFAD`	**Pflicht.** Eingabe: bei `--IN-BIN` die Textdatei, bei `--IN-TXT` der Lochstreifen (jeweils auch ein Verzeichnis mit Dateien)
`--IN-BIN`	Text → binär codierter R300-Lochstreifen. **Vorgabe**, darf entfallen
`--IN-TXT`	Umkehrung: binär codierter R300-Lochstreifen → Text. Nicht zusammen mit `--IN-BIN`
`--version`	Versionsnummer ausgeben
`-h`, `--help`	Hilfe ausgeben

`--LS-STANZER` und `--LS-LESER` müssen **beide** angegeben werden. Die Schreibweise mit Kleinbuchstaben (`--ls-stanzer`, `--ls-leser`, `--in-bin`, `--in-txt`) wird ebenfalls angenommen, ebenso die früheren Namen `--LESER`/`--STANZER` (und `--leser`/`--stanzer`); Abkürzungen der Optionen nicht.

Beispiele:

```
python3 c8205_lochstreifen_umwandlung.py --LS-STANZER band.txt --LS-LESER programm_text.txt
python3 c8205_lochstreifen_umwandlung.py --LS-STANZER virtuelle_Geraete/Leser1/ --LS-LESER hallo.txt
python3 c8205_lochstreifen_umwandlung.py --IN-TXT --LS-LESER band.txt --LS-STANZER band_klartext.txt
```

Unter Windows ggf. `py` oder `python` statt `python3` verwenden:

```
py c8205_lochstreifen_umwandlung.py --LS-STANZER band.txt --LS-LESER programm_text.txt
```

Beispiel: Die Textdatei enthält die Zeichen `a1 X` und einen Zeilenumbruch. Das Programm schreibt (Erklärung jeweils rechts, sie steht nicht in der Datei):

```
00110001     a            (Klbuch, Anfangslage, keine Umschaltung nötig)
00000001     1
00011010     Leertaste/ZR (in beiden Lagen gleich)
00101111     Um. GB       (X braucht Gbuch)
01100111     X
10111010     SatzMarke/WRZL
```

Mit `--IN-TXT` oder der Anzeige lässt sich das Ergebnis prüfen: `python3 c8205_lochstreifen_anzeige.py --LS-LESER band.txt --text`.

Meldungen: Am Ende steht eine Zusammenfassung auf der Standardausgabe (Zahl der Lochungszeilen, Zeichen, Umschaltungen, übersprungenen Zeichen). Bei `--IN-BIN` werden übersprungene Zeichen und eine leere Ausgabe als `Warnung:` auf der Fehlerausgabe gemeldet (höchstens 10 verschiedene Zeichen einzeln), bei `--IN-TXT`

übersprungene Lochungszeilen. Rückgabewert 0 = in Ordnung (auch mit Warnungen), 2 = Aufruf-, Lese- oder Schreibfehler (fehlende Option, `--LS-LESER` nicht gefunden, gleiche Datei für Ein- und Ausgabe, Schreibfehler).

4.4 Typischer Ablauf

1. Programm mit dem Interpreter ausführen, der über `--stanzer1 band.txt` einen Lochstreifen schreibt.
2. Mit der Anzeige ansehen: `--LS-LESER band.txt`.
3. Oder mit der Eingabe einen Lochstreifen erzeugen und dem Interpreter mit `--leser1 eingabe.txt` übergeben.
4. Oder einen fertigen Text mit der Umwandlung in einen Lochstreifen verwandeln (`--LS-LESER text.txt --LS-STANZER band.txt`) und dem Interpreter mit `--leser1 band.txt` übergeben.
5. Ein vollständiges Beispiel (Lochstreifen erzeugen, im TEST-Interpreter ein Programm auf die Trommel laden, danach im GRAFIK-Interpreter ausführen) steht in `C8205-Interpreter-Durchführung_eines_Tests.md`, Abschnitt 2h.

5. Technische Voraussetzungen (Installation siehe 5a)

- **Python 3** (getestet mit Python 3; nötig sind f-Strings und `argparse`, daher mindestens 3.6).
- Nur **Standardbibliothek**: `argparse`, `os`, `sys`, `tkinter`, außerdem `ctypes` und `time` für die Windows-Zwischenablage. Es ist nichts nachzuinstallieren. Die **Umwandlung** braucht nur `argparse`, `os` und `sys`, also weder Tkinter noch eine grafische Oberfläche; sie läuft auch auf einem Rechner ohne Fenstersystem.
- **Tkinter** für die Fenster:
  - Linux: Paket `python3-tk` (z. B. `sudo apt install python3-tk`).
  - Windows und macOS: im Python-Installer von python.org enthalten.
- **Grafische Oberfläche**: unter Linux muss `DISPLAY` oder `WAYLAND_DISPLAY` gesetzt sein. Fehlt sie oder fehlt Tkinter, geben beide Programme eine Warnung aus und weichen auf den Textmodus (`--text`) aus.
- **Zeichenkodierung**: Lochstreifendateien sind UTF-8; die Standardausgabe der Anzeige wird auf UTF-8 umgestellt.
- **Virtuelle Maschine:** Der Abgleich der Zwischenablage mit dem Host braucht das Gast-Zusatzprogramm (VirtualBox: Gasterweiterungen, QEMU/SPICE: `spice-vdagent`, VMware: `open-vm-tools`). Das Programm legt den Text nur in der Zwischenablage des Gastes ab; wie sie zum Host kommt, entscheidet dieses Zusatzprogramm.
- **Optional unter Linux**, für zuverlässiges Kopieren: Wayland `wl-clipboard` (`sudo apt install wl-clipboard`), X11 `xsel` (`sudo apt install xsel`) oder `xclip` (`sudo apt install xclip`). Ohne diese Programme wird nur die Tk-Zwischenablage verwendet.
- **Schreibrechte** für den Zielpfad (Eingabe und Umwandlung).
- Betriebssysteme: Linux (getestet), Windows (Zwischenablage-Zweig noch ungetestet), macOS (nicht getestet, Tk-Verfahren wie unter Linux). Die Umwandlung ist plattformneutral (keine Systemaufrufe außer Dateizugriff) und unter Linux getestet; ein Lauf unter Windows steht noch aus (Abschnitt 6, Punkt 5).

5a. Installation der benötigten Programme

Benötigt wird nur Python 3 mit Tkinter. Die Zwischenablage-Programme sind optional (nur Linux/WSL). Die beiden `.py`-Dateien selbst müssen nicht installiert werden, sie laufen direkt.

Linux

Python 3 ist meist vorhanden (Prüfung: `python3 --version`).

Distribution	Python und Tkinter	Zwischenablage (optional)
Debian, Ubuntu, Mint	`sudo apt install python3 python3-tk`	X11: `sudo apt install xsel xclip`; Wayland: `sudo apt install wl-clipboard`
Fedora	`sudo dnf install python3 python3-tkinter`	X11: `sudo dnf install xsel xclip`; Wayland: `sudo dnf install wl-clipboard`
Arch, Manjaro	`sudo pacman -S python tk`	X11: `sudo pacman -S xsel xclip`; Wayland: `sudo pacman -S wl-clipboard`
openSUSE	`sudo zypper install python3 python3-tk`	X11: `sudo zypper install xsel xclip`; Wayland: `sudo zypper install wl-clipboard`

Prüfung, ob Tkinter da ist: `python3 -m tkinter` öffnet ein kleines Testfenster. Ob X11 oder Wayland läuft, zeigt `echo $XDG_SESSION_TYPE`.

Windows

1. Python 3 von <https://www.python.org/downloads/> herunterladen und installieren. Im Installer **„Add python.exe to PATH“** ankreuzen. Die Option **„tcl/tk and IDLE“** ist vorgegeben und muss angekreuzt bleiben.
2. Prüfung in der Eingabeaufforderung oder PowerShell: `py --version` und `py -m tkinter` (öffnet ein Testfenster).
3. Aufruf mit `py c8205_lochstreifen_anzeige.py --LS-LESER band.txt`.
4. Weitere Programme sind nicht nötig: Die Zwischenablage läuft über die Win32-API, `ctypes` gehört zu Python.

Hinweis: Die Python-Version aus dem Microsoft Store enthält ebenfalls Tkinter. Bei einer abgespeckten Installation ohne Tcl/Tk Python über „Ändern“ im Installer nachrüsten.

macOS

Python 3 von <https://www.python.org/downloads/> installieren (enthält Tkinter), Aufruf mit `python3`. `pbcopy` ist im System vorhanden. Die Python-Version von Homebrew braucht zusätzlich `brew install python-tk`.

WSL (Windows-Subsystem für Linux)

Wie Linux (Debian/Ubuntu: `sudo apt install python3 python3-tk`). Fenster brauchen WSLg (Windows 11 oder aktuelles Windows 10). Die Zwischenablage zum Windows-Host geht über `clip.exe`, das in WSL schon vorhanden ist.

Virtuelle Maschinen (Abgleich der Zwischenablage mit dem Host)

Umgebung	Installation im Gast (Debian/Ubuntu)
VMware	`sudo apt install open-vm-tools open-vm-tools-desktop`, danach Neustart
VirtualBox	Gasterweiterungen aus dem Menü „Geräte“ einlegen und installieren; Zwischenablage: Geräte, Gemeinsame Zwischenablage, bidirektional
QEMU/SPICE	`sudo apt install spice-vdagent`

---

6. Test unter Windows (noch offen)

Zu prüfen, sobald ein Windows-Rechner verfügbar ist:

1. Anzeige mit einer beliebigen Lochstreifendatei starten, Text mit der Maus markieren und in einem anderen Programm (z. B. Editor) einfügen.
2. Dasselbe mit Strg+C, Strg+Einfg und dem Kontextmenü.
3. Im Eingabefenster `@`, `{`, `}`, `[`, `]`, `\`, `~`, `|` per AltGr tippen (soweit im R300-Kode vorhanden, sonst Signalton) und prüfen, dass sie nicht als Steuerbefehl wirken.
4. Bei Problemen mit `--debug` starten und die Ausgabe auswerten; dort steht, ob das Kopieren per `Win32` oder per `Tk` lief.
5. Umwandlung: `py c8205_lochstreifen_umwandlung.py --LS-STANZER band.txt --LS-LESER text.txt` mit einer Textdatei mit Windows-Zeilenenden (`\r\n`) ausführen und prüfen, dass `band.txt` je Zeilenende genau eine SatzMarke (`10111010`) enthält, keine Leerzeilen und keine Reste von `\r`.
6. Umwandlung `--IN-TXT`: Lochstreifen mit `--IN-BIN` erzeugen und mit `--IN-TXT` zurückwandeln; die Textdatei muss (bis auf `\r`) dem Original entsprechen.